Trie

- Trie Introduction
- Why Trie?
- TrieNode Design
- Basic Implementation
- Insert
- Search
- StartsWith
- Complexity
- Dry Runs
- LeetCode 208 Solution

Part 2

- Delete Operation
- Count Words
- Count Prefixes
- Longest Prefix Match
- Complete Trie Template
- Interview Questions

Part 3

- DFS Traversal
- Print Dictionary
- Auto Complete
- Lexicographical Order
- Word Suggestions
- Memory Optimizations

Part 4

- Array Trie vs HashMap Trie
- Compressed Trie (Radix Tree)
- Binary Trie

- XOR Problems
- Maximum XOR Pair

Part 5

- LeetCode Problems
- LC 211
- LC 648
- LC 677
- LC 720
- LC 820
- LC 1268
- LC 421
- LC 1707

Part 6

- Interview Cheat Sheet
- Common Mistakes
- Complexity Tables
- Reusable Templates
- Practice Problems
- Revision Notes

Each part will be **100–150+ lines**, resulting in a **600+ line guide**.

PART 1 — Trie (Prefix Tree) Fundamentals

---

1. What is a Trie?

A **Trie** (pronounced *try*) is a tree-based data structure used to store **strings** efficiently.

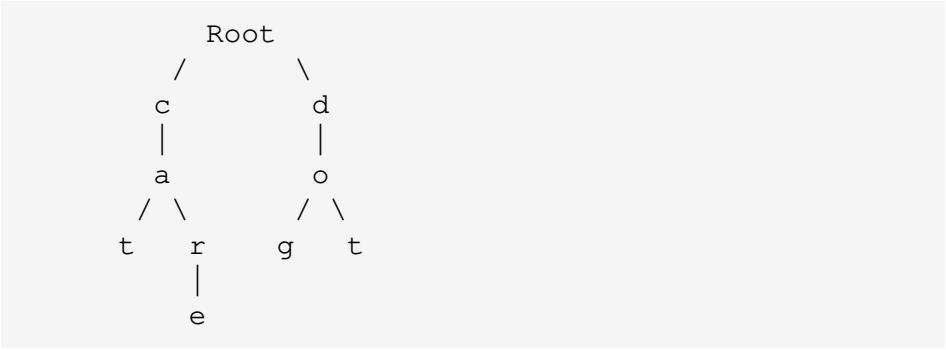
Instead of storing complete strings in each node, every node stores **one character**.

Words having common prefixes share the same path.

Example:

```
Words:

cat
car
care
dog
dot
```

Trie:



Notice

```
cat
car
care
```

share

```
c → a
```

Only one copy of prefix is stored.

This is why Trie is called a **Prefix Tree**.

---

2. Why not HashMap?

Suppose we store

```
apple
application
apply
apt
```

A HashMap stores

```
apple
application
apply
apt
```

individually.

Trie stores

```
a
|
p
|
p
|\
l t
|
```

```
e
|
i...
```

Common prefixes are shared.

---

3. Applications of Trie

Very common interview uses include:

- Dictionary
- Spell Checker
- Auto Complete
- Search Suggestions
- IP Routing
- Contacts List
- Prefix Search
- Longest Prefix Matching
- Word Games
- Phone Directory
- Maximum XOR (Binary Trie)

Popular companies asking Trie:

- Google
- Microsoft
- Amazon
- Adobe
- Atlassian
- Walmart
- Uber
- Oracle

---

4. Time Complexity

Operation	Complexity
Insert	O(L)
Search	O(L)
StartsWith	O(L)
Delete	O(L)
Prefix Count	O(L)

where

```
L = length of word
```

Unlike BST,

Trie complexity depends on

```
characters

NOT

number of words.
```

---

5. Trie Node Design

Each node requires

```
26 children
```

and

```
isEndOfWord
```

Java Implementation

```
class TrieNode {

    TrieNode[] children;
    boolean isEnd;

    TrieNode() {
        children = new TrieNode[26];
        isEnd = false;
    }
}
```

Why 26?

```
'a' -> index 0

'b' -> index 1

...

'z' -> index 25
```

Conversion:

```
int index = ch - 'a';
```

Example

```
'a' - 'a' = 0

'b' - 'a' = 1

'z' - 'a' = 25
```

This gives

```
O(1)
```

child access.

---

6. Trie Class Skeleton

```
class Trie {

    private TrieNode root;

    public Trie() {
        root = new TrieNode();
    }
}
```

Every operation starts from

```
root
```

---

7. Insert Operation

Insert

```
cat
```

Initially

```
Root
```

Step 1

```
Root

 |
 c
```

Step 2

```
Root

 |
 c
 |
 a
```

Step 3

```
Root

 |
 c
 |
 a
 |
 t*
```

'*' means

```
End of word
```

Insert Algorithm

For every character

```
If child absent

Create it

Move to child

Repeat

Finally

Mark end=true
```

Java Code

```
class Trie {

    private TrieNode root;

    public Trie() {
        root = new TrieNode();
    }

    public void insert(String word) {

        TrieNode current = root;
```

```
        for (int i = 0; i < word.length(); i++) {

            char ch = word.charAt(i);

            int index = ch - 'a';

            if (current.children[index] == null) {
                current.children[index] = new TrieNode();
            }

            current = current.children[index];
        }

        current.isEnd = true;
    }
}
```

Dry Run

Insert

```
cat
```

Iteration 1

```
Character = c

index = 2

create node
```

Iteration 2

```
Character = a
```

```
index = 0

create node
```

Iteration 3

```
Character = t

index = 19

create node
```

Finally

```
isEnd = true
```

Trie becomes

```
root
 |
c
|
a
|
t*
```

---

8. Insert Another Word

Insert

```
car
```

Already exists

```
c

a
```

Only

```
r
```

gets created.

Final

```
            root
              |
              c
              |
              a
             /   \
           t*     r*
```

Shared prefix saves memory.

---

9. Search Operation

Goal

```
Return true
```

```
only if
complete word exists.
```

Searching

```
cat
```

Traversal

```
root
↓
c
↓
a
↓
t
```

Reached

```
isEnd=true
return true
```

Searching

```
ca
```

Traversal succeeds.

But

```
isEnd=false
```

Return

```
false
```

because

```
ca
```

is only a prefix.

Search Algorithm

For each character

```
Find child

If absent

return false

Else

move ahead

Finally

return isEnd
```

Java Code

```
public boolean search(String word) {

    TrieNode current = root;

    for (int i = 0; i < word.length(); i++) {

        int index = word.charAt(i) - 'a';

        if (current.children[index] == null) {
            return false;
        }

        current = current.children[index];
    }

    return current.isEnd;
}
```

Dry Run

Trie

```
cat

car
```

Search

```
cat
```

Visit

```
c
↓
a
↓
t
```

Reached

```
isEnd=true
```

Answer

```
true
```

Search

```
cab
```

Visit

```
c
↓
a
↓
b
```

No child

```

```

```
return false
```

---

10. StartsWith Operation

Unlike search,

we only check whether

```
prefix exists.
```

Example

```
cat

car

care
```

Queries

```
ca

true
```

```
car

true
```

```
care

true
```

```
caref

false
```

Algorithm

Traverse normally.

If traversal finishes,

return

```
true
```

No need to check

```
isEnd
```

Java Code

```
public boolean startsWith(String prefix) {

    TrieNode current = root;

    for (int i = 0; i < prefix.length(); i++) {

        int index = prefix.charAt(i) - 'a';

        if (current.children[index] == null) {
            return false;
        }
```

```
            current = current.children[index];
        }

        return true;
    }
```

Dry Run

Words

```
apple

application

apply
```

Query

```
app
```

Traversal

```
a

↓

p

↓

p
```

Traversal completed.

Answer

```

```

```
true
```

---

11. LeetCode 208 — Implement Trie (Prefix Tree)

This is the foundational Trie problem and introduces the three core operations:

- `insert`
- `search`
- `startsWith`

LeetCode-Compatible Java Solution

```
class Trie {

    class TrieNode {

        TrieNode[] children;
        boolean isEnd;

        TrieNode() {
            children = new TrieNode[26];
            isEnd = false;
        }
    }

    private TrieNode root;

    public Trie() {
        root = new TrieNode();
    }
```

```
    public void insert(String word) {

        TrieNode current = root;

        for (int i = 0; i < word.length(); i++) {

            int index = word.charAt(i) - 'a';

            if (current.children[index] == null) {
                current.children[index] = new TrieNode();
            }

            current = current.children[index];
        }

        current.isEnd = true;
    }

    public boolean search(String word) {

        TrieNode current = root;

        for (int i = 0; i < word.length(); i++) {

            int index = word.charAt(i) - 'a';

            if (current.children[index] == null) {
                return false;
            }

            current = current.children[index];
        }

        return current.isEnd;
    }

    public boolean startsWith(String prefix) {

        TrieNode current = root;
```

```
        for (int i = 0; i < prefix.length(); i++) {

            int index = prefix.charAt(i) - 'a';

            if (current.children[index] == null) {
                return false;
            }

            current = current.children[index];
        }

        return true;
    }
}
```

PART 2 — Advanced Trie Operations

In Part 1, we learned:

- TrieNode
- Insert
- Search
- StartsWith
- LeetCode 208

Now we'll extend the Trie to solve more interview problems.

---

12. Storing Additional

Information in Each Node

A Trie node can store much more than just children and `isEnd`.

Common additions:

```
children
isEnd
wordCount
prefixCount
frequency
word
```

Different interview problems require different node designs.

Example:

```
class TrieNode {

    TrieNode[] children;

    boolean isEnd;

    int prefixCount;

    int wordCount;

    TrieNode() {

        children = new TrieNode[26];

        isEnd = false;

        prefixCount = 0;

        wordCount = 0;
    }
```

```
}
```

Why prefixCount?

Suppose we insert

```
cat
car
care
can
```

Trie

```
root
 |
 c(4)
 |
 a(4)
 |
 +----------------------+
 |   |         |        |
 t   r(2)      n        ...
     |
     e
```

Numbers indicate

```
prefixCount
```

Node

```
c
```

has

```
4 words
```

passing through.

Why wordCount?

Suppose

```
cat
cat
cat
```

are inserted.

Instead of

```
isEnd=true
```

store

```
wordCount=3
```

Useful for

- Duplicate words
- Frequency dictionaries
- Autocomplete ranking

---

13. Insert with Prefix Count

Instead of only creating nodes,

increase prefix count.

Algorithm

```
Move character

Create node if absent

Move to child

Increase prefixCount

Repeat

Increase wordCount
```

Java Code

```
class Trie {

    class TrieNode {

        TrieNode[] children;

        int prefixCount;

        int wordCount;

        TrieNode() {
```

```
            children = new TrieNode[26];

            prefixCount = 0;

            wordCount = 0;
        }
    }

    TrieNode root;

    Trie() {
        root = new TrieNode();
    }

    public void insert(String word) {

        TrieNode current = root;

        for (int i = 0; i < word.length(); i++) {

            int index = word.charAt(i) - 'a';

            if (current.children[index] == null) {
                current.children[index] = new TrieNode();
            }

            current = current.children[index];

            current.prefixCount++;
        }

        current.wordCount++;
    }
}
```

Dry Run

Insert

```
cat
```

Counts become

```
c = 1

a = 1

t = 1
```

Insert

```
car
```

Now

```
c = 2

a = 2

t = 1

r = 1
```

Insert

```
care
```

Now

```
c = 3

a = 3

r = 2
```

```
e = 1
```

---

14. Count Words Equal To

Interview question

> How many times was a word inserted?

Example

```
cat
cat
cat
car
```

Query

```
countWordsEqualTo("cat")
```

Answer

```
3
```

Algorithm

Traverse.

If path missing

```

```

```
return 0
```

Otherwise

```
return wordCount
```

Java Code

```
public int countWordsEqualTo(String word) {

    TrieNode current = root;

    for (int i = 0; i < word.length(); i++) {

        int index = word.charAt(i) - 'a';

        if (current.children[index] == null) {
            return 0;
        }

        current = current.children[index];
    }

    return current.wordCount;
}
```

Example

Inserted

```
cat
```

```
cat
cat
car
```

Queries

```
countWordsEqualTo("cat")
```

Answer

```
3
```

```
countWordsEqualTo("car")
```

Answer

```
1
```

```
countWordsEqualTo("care")
```

Answer

```
0
```

Complexity

```
O(L)
```

---

15. Count Words Starting With Prefix

Very common interview question.

Example

Inserted

```
apple

app

application

apply

apt

bat
```

Query

```
app
```

Answer

```
4
```

because

```
```

```
app

apple

application

apply
```

start with

```
app
```

Algorithm

Traverse till prefix.

Return

```
prefixCount
```

Java Code

```
public int countWordsStartingWith(String prefix) {

    TrieNode current = root;

    for (int i = 0; i < prefix.length(); i++) {

        int index = prefix.charAt(i) - 'a';

        if (current.children[index] == null) {
```

```
            return 0;
        }

        current = current.children[index];
    }

    return current.prefixCount;
}
```

Example

Words

```
car

care

careful

careless

cat

dog
```

Query

```
car
```

Answer

```
4
```

---

16. Erase/Delete a Word

This is one of the most misunderstood Trie operations.

Important observation

Deleting

```
care
```

must NOT delete

```
car
```

because

```
car
```

still exists.

Suppose

```
car

care
```

Trie

```
root

|

c

|
```

```
a
|
r*
|
e*
```

Delete

```
care
```

Only

```
e
```

should disappear.

The path

```
c
a
r
```

must remain.

Approach 1 (Most Common)

Decrease

```
prefixCount
```

Decrease

```
wordCount
```

Remove unused nodes.

Delete Algorithm

Step 1

Search word.

If absent

```
return
```

Step 2

Traverse again.

Decrease

```
prefixCount
```

Step 3

Decrease

```

```

```
wordCount
```

Step 4

If

```
prefixCount==0
```

remove node.

Java Recursive Delete

```
public void erase(String word) {

    erase(root, word, 0);
}

private boolean erase(TrieNode node, String word, int dep

    if (node == null) {
        return false;
    }

    if (depth == word.length()) {

        if (node.wordCount > 0) {
            node.wordCount--;
        }

        return node.wordCount == 0 && isEmpty(node);
    }

    int index = word.charAt(depth) - 'a';
```

```
    TrieNode child = node.children[index];

    if (child == null) {
        return false;
    }

    boolean shouldDeleteChild = erase(child, word, depth

    if (shouldDeleteChild) {
        node.children[index] = null;
    }

    return node.wordCount == 0 && isEmpty(node);
}
```

Helper Function

```
private boolean isEmpty(TrieNode node) {

    for (int i = 0; i < 26; i++) {

        if (node.children[i] != null) {
            return false;
        }
    }

    return true;
}
```

Complexity

```
Time : O(L)

Space : O(L)
```

because of recursion.

Interview Note

Many interviewers accept

```
wordCount--

prefixCount--
```

without physically deleting nodes.

Reason

Most production systems prefer

Lazy Deletion

instead of

Physical Deletion.

---

17. Longest Prefix Match

Frequently asked in networking interviews.

Example

Dictionary

```
there

their

then

the
```

Query

```
thereafter
```

Longest matching prefix

```
there
```

Example

Dictionary

```
cat

care

dog
```

Search

```
carefully
```

Answer

```
care
```

Algorithm

Keep traversing.

Whenever

```
wordCount>0
```

save current position.

Continue until path breaks.

Return last valid word.

Java Code

```
public String longestPrefix(String word) {

    TrieNode current = root;

    StringBuilder answer = new StringBuilder();

    String longest = "";

    for (int i = 0; i < word.length(); i++) {

        int index = word.charAt(i) - 'a';

        if (current.children[index] == null) {
            break;
        }

        current = current.children[index];
```

```
            answer.append(word.charAt(i));

            if (current.wordCount > 0) {
                longest = answer.toString();
            }
        }

        return longest;
    }
```

Example

Dictionary

```
car

care

careful

cat
```

Input

```
carefully
```

Traversal

```
c

↓

a

↓
```

```
r

↓

e

↓

f

↓

u
```

Valid words

```
car

care

careful
```

Answer

```
careful
```

---

18. Complete Interview Trie Template

```
class TrieNode {

    TrieNode[] children;
```

```
    int prefixCount;

    int wordCount;

    TrieNode() {

        children = new TrieNode[26];

        prefixCount = 0;

        wordCount = 0;
    }
}
```

Trie

```
class Trie {

    TrieNode root;

    Trie() {
        root = new TrieNode();
    }

    public void insert(String word) { }

    public boolean search(String word) { }

    public boolean startsWith(String prefix) { }

    public int countWordsEqualTo(String word) { }

    public int countWordsStartingWith(String prefix) { }

    public void erase(String word) { }

    public String longestPrefix(String word) { }
}
```

This template forms the basis for many advanced Trie interview questions.

---

19. Common Interview Questions

Easy

- Implement Trie
- Search Word
- Starts With
- Count Words
- Prefix Count

Medium

- Delete Word
- Longest Prefix Match
- Auto Complete
- Search Suggestions
- Replace Words

Hard

- Word Search II
- Maximum XOR
- Stream Checker
- Prefix and Suffix Search
- Design Search Autocomplete System

Complexity Summary

Operation	Time	Space
Insert	O(L)	O(L) new nodes
Search	O(L)	O(1)
StartsWith	O(L)	O(1)
Count Words	O(L)	O(1)
Count Prefix	O(L)	O(1)
Longest Prefix	O(L)	O(1)
Delete	O(L)	O(L) recursive

PART 3 — Trie Traversal, DFS, Auto Complete and Lexicographical Order

In the previous parts we learned how to

- Insert
- Search
- StartsWith
- Delete
- Count Words
- Count Prefixes
- Longest Prefix

Now we'll learn how to **traverse a Trie**, which is the foundation for problems like:

- Auto Complete
- Search Suggestions
- Dictionary Printing
- Lexicographical Ordering
- Word Search
- Search Engines

---

20. Traversing a Trie

Unlike a Binary Tree,

Trie nodes can have

```
26 children
```

instead of

```
2 children
```

Traversal therefore becomes

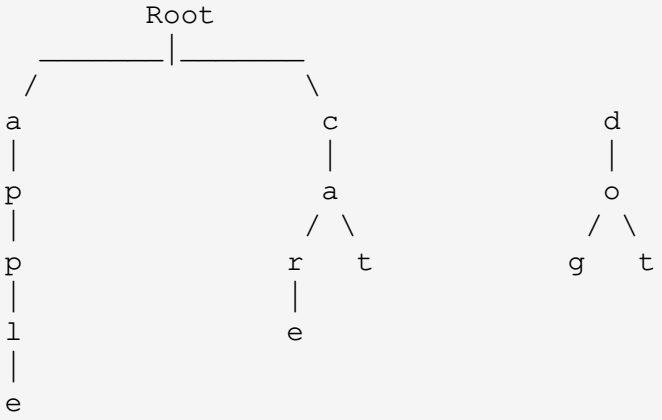
```
For every child

Visit child

Continue recursively
```

Suppose we inserted

```
cat
car
care
```

```
dog
dot
apple
```

Trie



A DFS traversal visits every node.

---

21. Why DFS?

Almost every advanced Trie problem uses DFS.

Examples

- Print all words
- Search suggestions
- Auto Complete
- Lexicographical Dictionary
- Find longest word
- Word Search II

Whenever you hear

> Return all matching words

think

```
DFS
```

---

22. Printing Every Word in Trie

Suppose Trie contains

```
apple

apt

bat

ball

cat
```

Output

```
apple

apt

ball

bat

cat
```

Notice

Words naturally appear in

```
Alphabetical Order
```

because children are visited from

```
a

↓

z
```

Algorithm

Maintain

```
currentWord
```

Whenever

```
isEnd == true
```

print

```
currentWord
```

Continue DFS.

Java Code

```
public void printAllWords() {

    StringBuilder current = new StringBuilder();

    dfs(root, current);
}

private void dfs(TrieNode node, StringBuilder current) {

    if (node == null) {
        return;
    }

    if (node.wordCount > 0) {
        System.out.println(current.toString());
    }

    for (int i = 0; i < 26; i++) {

        if (node.children[i] != null) {

            current.append((char) ('a' + i));

            dfs(node.children[i], current);

            current.deleteCharAt(current.length() - 1);
        }
    }
}
```

Dry Run

Words

```
cat

car

care
```

Traversal

```
root

↓

c

↓

a

↓

r
```

Reached

```
wordCount>0
```

Print

```
car
```

Continue

```
↓

e
```

Print

```

```

```
care
```

Backtrack

Visit

```
t
```

Print

```
cat
```

Output

```
car

care

cat
```

Why Backtracking?

Suppose current word is

```
care
```

DFS finishes.

Need to return to

```
car
```

before exploring another branch.

Hence

```
current.deleteCharAt(current.length()-1);
```

This is identical to

- Binary Tree Path
- N Queens
- Sudoku
- Maze

Backtracking always

```
Undo

↓

Explore another choice
```

---

23. Returning Words Instead of Printing

Interviewers usually ask

> Return a list.

Java

```
public List<String> getAllWords() {
```

```
    List<String> result = new ArrayList<>();

    dfs(root, new StringBuilder(), result);

    return result;
}

private void dfs(
        TrieNode node,
        StringBuilder current,
        List<String> result) {

    if (node == null) {
        return;
    }

    if (node.wordCount > 0) {
        result.add(current.toString());
    }

    for (int i = 0; i < 26; i++) {

        if (node.children[i] != null) {

            current.append((char) ('a' + i));

            dfs(node.children[i], current, result);

            current.deleteCharAt(current.length() - 1);
        }
    }
}
```

Output

```
[
apple,
apt,
```

```
ball,
bat,
cat
]
```

Complexity

Let

```
N = number of Trie nodes
```

Traversal

```
Time = O(N)

Space = O(H)

H = maximum word length
```

---

24. Lexicographical Ordering

One beautiful property of Trie

It naturally stores words

in

```
Dictionary Order
```

Example

Inserted randomly

```
dog

apple

banana

cat

ball
```

Printing Trie

always gives

```
apple

ball

banana

cat

dog
```

No sorting required.

Why?

Because

```
for(int i=0;i<26;i++)
```

visits

```
a

↓

b

↓

c

↓

...

↓

z
```

Interview Tip

Many candidates sort

```
Collections.sort()
```

Unnecessary.

Trie already stores

alphabetically.

---

25. Auto Complete

One of Google's most famous interview topics.

Suppose dictionary

```
apple

application

apply

app

apt

banana
```

User types

```
app
```

Suggestions

```
app

apple

application

apply
```

Steps

Step 1

Reach prefix node

```

```

```
app
```

Step 2

DFS from there

Collect words.

Visualization

```
app
 |
 +------+
 |      |
 l      ...
 |
 e

↓

apple
```

Java Code

```
public List<String> autoComplete(String prefix) {

    List<String> result = new ArrayList<>();

    TrieNode current = root;

    for (int i = 0; i < prefix.length(); i++) {

        int index = prefix.charAt(i) - 'a';
```

```
            if (current.children[index] == null) {
                return result;
            }

            current = current.children[index];
        }

        dfs(current,
            new StringBuilder(prefix),
            result);

        return result;
    }
```

Notice

DFS begins

at

```
prefix node
```

instead of

```
root
```

Example

Dictionary

```
car

care

careful

cargo
```

```
cat

dog
```

Query

```
car
```

Output

```
car

care

careful

cargo
```

Complexity

Finding prefix

```
O(L)
```

DFS

```
O(K)
```

where

```
K
```

```
=

characters visited
```

Overall

```
O(L + K)
```

---

26. Limiting Suggestions

Search engines rarely return

every word.

Usually

Top 3

or

Top 5

suggestions.

Java

```
private void dfs(
        TrieNode node,
        StringBuilder current,
        List<String> result) {

    if (node == null) {
        return;
```

```
        }

        if (result.size() == 3) {
            return;
        }

        if (node.wordCount > 0) {
            result.add(current.toString());
        }

        for (int i = 0; i < 26; i++) {

            if (node.children[i] != null) {

                current.append((char) ('a' + i));

                dfs(node.children[i], current, result);

                current.deleteCharAt(current.length() - 1);
            }
        }
    }
```

This pattern is exactly used in

Search Suggestion Systems.

---

27. LeetCode 1268 Pattern

Problem

Search Suggestions System

Dictionary

```
mobile

mouse

moneypot

monitor

mousepad
```

Search

```
mouse
```

Expected

For every prefix

```
m

mo

mou

mous

mouse
```

return

Top 3 suggestions.

Algorithm

For each prefix

```
Find node
```

```
↓
DFS
↓
Collect first 3 words
```

Exactly the same

Auto Complete algorithm.

---

28. Finding the Longest Word in Trie

Dictionary

```
banana
cat
careful
application
```

Answer

```
application
```

Simple DFS

Maintain

```
```

```
bestWord
```

Whenever

```
current.length()

>

bestWord.length()
```

replace answer.

Java

```
private String longestWord = "";

private void dfsLongest(
        TrieNode node,
        StringBuilder current) {

    if (node == null) {
        return;
    }

    if (node.wordCount > 0 &&
            current.length() > longestWord.length()) {

        longestWord = current.toString();
    }

    for (int i = 0; i < 26; i++) {

        if (node.children[i] != null) {

            current.append((char) ('a' + i));

            dfsLongest(node.children[i], current);
```

```
                current.deleteCharAt(current.length() - 1);
            }
        }
    }
```

---

29. Counting Total Words

Instead of

DFS

count every

```
wordCount>0
```

node.

Java

```
public int totalWords() {

    return count(root);
}

private int count(TrieNode node) {

    if (node == null) {
        return 0;
    }

    int total = node.wordCount;
```

```
    for (int i = 0; i < 26; i++) {

        total += count(node.children[i]);
    }

    return total;
}
```

Example

Words

```
cat

car

dog

apple
```

Answer

```
4
```

---

30. Finding Total Trie Nodes

Useful interview question.

```
public int totalNodes() {

    return nodes(root);
}
```

```
private int nodes(TrieNode node) {

    if (node == null) {
        return 0;
    }

    int answer = 1;

    for (int i = 0; i < 26; i++) {
        answer += nodes(node.children[i]);
    }

    return answer;
}
```

---

31. Recursive vs Iterative DFS

Recursive

Advantages

- Easy
- Clean
- Short

Disadvantages

- Stack usage
- Recursion depth

Iterative

Advantages

- No recursion

- Better control

Disadvantages

- Slightly longer code

Interview preference

Recursive DFS

is almost always accepted.

---

32. Common DFS Mistakes

Mistake 1

Forgetting

```
deleteCharAt()
```

Result

Wrong words.

Mistake 2

Printing before appending character.

Wrong

```
ca
```

instead of

```
cat
```

Mistake 3

Not checking

```
wordCount > 0
```

Every prefix becomes a word.

Mistake 4

Returning after first child.

Wrong

```
apple
```

Correct

```
apple

application

apply
```

Need to visit

all

children.

---

33. Interview Problems Based on DFS

Easy

- Print Dictionary
- Count Words

Medium

- Search Suggestions
- Auto Complete
- Longest Word
- Replace Words

Hard

- Word Search II
- Design Search Autocomplete
- Stream Checker
- Prefix Search Engine

PART 4 — Advanced Trie Implementations (Array Trie, HashMap Trie, Compressed Trie, Binary Trie & XOR Problems)

In the previous parts, we built a standard Trie that stores lowercase

English words.

In interviews, however, you may encounter situations where:

- The alphabet size is unknown
- Characters include uppercase, digits, or Unicode
- Memory usage becomes important
- The Trie stores binary numbers instead of words

This part covers those advanced variations.

---

34. Array Trie vs HashMap Trie

Our previous implementation used

```
TrieNode[] children = new TrieNode[26];
```

Every node always allocates

```
26 references
```

even if only one child exists.

Example

Only one word inserted

```
apple
```

Node

```
a
```

still allocates

```
26 child pointers
```

Only one is actually used.

Memory wasted

```
25 unused references
```

Array Trie

Implementation

```
class TrieNode {

    TrieNode[] children;

    boolean isEnd;

    TrieNode() {

        children = new TrieNode[26];

        isEnd = false;
    }
}
```

Advantages

- Fastest lookup
- Constant-time child access
- Very simple implementation
- Best for lowercase English letters

Disadvantages

- Wastes memory
- Fixed alphabet
- Cannot easily support Unicode

Time Complexity

Operation	Complexity
Child Access	O(1)
Insert	O(L)
Search	O(L)

Memory

```
26 pointers per node
```

---

35. HashMap Trie

Instead of allocating

```
26 children
```

store only existing children.

Implementation

```
import java.util.HashMap;
```

```
class TrieNode {

    HashMap<Character, TrieNode> children;

    boolean isEnd;

    TrieNode() {

        children = new HashMap<>();

        isEnd = false;
    }
}
```

Visualization

Instead of

```
[a][b][c][d]...[z]
```

store only

```
a

↓

p
```

Advantages

- Saves memory
- Supports Unicode
- Supports digits
- Supports symbols
- Dynamic alphabet

Disadvantages

- Slightly slower
- HashMap overhead

Complexity

Operation	Complexity
Access	Average O(1)
Insert	O(L)
Search	O(L)

Array vs HashMap

Feature	Array Trie	HashMap Trie
Lookup Speed	⭐⭐⭐⭐⭐	⭐⭐⭐⭐
Memory Usage	⭐⭐	⭐⭐⭐⭐⭐
Unicode	✗	✓
Simplicity	⭐⭐⭐⭐⭐	⭐⭐⭐
Interview Frequency	Very High	High

Interview Tip

If interviewer says

> Lowercase English only

Use

```
TrieNode[26]
```

If interviewer says

> Any character

Use

```
HashMap<Character, TrieNode>
```

---

36. HashMap Trie Insert

```
public void insert(String word) {

    TrieNode current = root;

    for (char ch : word.toCharArray()) {

        current.children.putIfAbsent(ch, new TrieNode());

        current = current.children.get(ch);
    }

    current.isEnd = true;
}
```

Without enhanced for-loop

```
public void insert(String word) {

    TrieNode current = root;

    for (int i = 0; i < word.length(); i++) {
```

```
        char ch = word.charAt(i);

        if (!current.children.containsKey(ch)) {

            current.children.put(ch, new TrieNode());
        }

        current = current.children.get(ch);
    }

    current.isEnd = true;
}
```

37. HashMap Trie Search

```
public boolean search(String word) {

    TrieNode current = root;

    for (int i = 0; i < word.length(); i++) {

        char ch = word.charAt(i);

        if (!current.children.containsKey(ch)) {

            return false;
        }

        current = current.children.get(ch);
    }

    return current.isEnd;
}
```

38. Compressed Trie (Radix Tree)

Standard Trie

stores

```
c

↓

a

↓

r

↓

e
```

Compressed Trie stores

```
care
```

as

one edge.

Example

Dictionary

```
care
```

```
careful
```

Standard Trie

```
c

↓

a

↓

r

↓

e

↓

f...
```

Compressed Trie

```
care

↓

ful
```

Advantage

Much fewer nodes.

Disadvantage

Insertion becomes more complex because edges may need splitting.

Where used?

- IP Routing
- File Systems
- Search Engines
- Databases

39. Ternary Search Trie (TST)

A TST combines ideas from a Trie and a BST.

Each node stores

```
character
```

and has

```
left

middle

right
```

children.

Visualization

```
          c
        /   |   \
       b    a    d
            |
            t
```

Advantages

- Less memory than Array Trie
- Faster than BST for prefix queries

Disadvantages

- Harder to implement
- Less common in coding interviews

---

40. Binary Trie

One of the most important advanced Trie topics.

Instead of storing

```
a-z
```

store

```
0

1
```

Every integer becomes

its binary representation.

Example

Number

```
13
```

Binary

```
1101
```

Trie

```
Root
|
1
|
1
|
0
|
1
```

Children

```
children[0]

children[1]
```

instead of

```
children[26]
```

Binary Trie Node

```
class TrieNode {

    TrieNode[] children;

    TrieNode() {

        children = new TrieNode[2];
    }
}
```

Why Binary Trie?

Interview problems

- Maximum XOR Pair
- Minimum XOR Pair
- IP Routing
- Network Prefix Matching
- LeetCode 421
- LeetCode 1707

---

41. Insert Number into Binary Trie

Instead of characters

iterate through bits.

Use

```
31

↓

0
```

because Java integers have

32 bits.

Java

```
public void insert(int num) {

    TrieNode current = root;

    for (int i = 31; i >= 0; i--) {

        int bit = (num >> i) & 1;

        if (current.children[bit] == null) {

            current.children[bit] = new TrieNode();
        }

        current = current.children[bit];
    }
}
```

Example

Insert

```
10
```

Binary

```
1010
```

Stored as

```
1

↓

0

↓

1

↓

0
```

---

42. Understanding XOR

Truth Table

A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Important observation

Maximum XOR occurs when

bits are

```
different
```

Example

```
0 XOR 1 = 1
```

Good.

```
1 XOR 1 = 0
```

Bad.

---

43. Maximum XOR Strategy

Suppose current bit

```
1
```

We want

```
0
```

because

```
1 XOR 0 = 1
```

Similarly

If current bit

```
0
```

choose

```
1
```

This greedy approach produces

maximum XOR.

---

44. Searching Maximum XOR

Java

```
public int getMaxXor(int num) {
    TrieNode current = root;
    int answer = 0;
    for (int i = 31; i >= 0; i--) {
        int bit = (num >> i) & 1;
        int opposite = 1 - bit;
        if (current.children[opposite] != null) {
            answer |= (1 << i);
            current = current.children[opposite];
```

```
        } else {

            current = current.children[bit];
        }
    }

    return answer;
}
```

Example

Numbers

```
3

10

5

25

2

8
```

Maximum XOR

```
5 XOR 25 = 28
```

Binary

```
00101

11001
```

Result

```
11100

=

28
```

---

45. LeetCode 421

Maximum XOR of Two Numbers in an Array

Problem

Given an array

```
nums
```

find

```
maximum XOR
```

Brute Force

```
for every pair

compute XOR
```

Complexity

```
O(N²)
```

Optimal

Use

```
Binary Trie
```

Complexity

```
Insert

O(32N)

Search

O(32N)

Total

O(N)
```

Since

```
32
```

is constant.

---

46. Binary Trie Template

```
class TrieNode {

    TrieNode[] children;

    TrieNode() {

        children = new TrieNode[2];
```

```
    }
}

class BinaryTrie {

    TrieNode root;

    BinaryTrie() {

        root = new TrieNode();
    }

    public void insert(int num) {

        TrieNode current = root;

        for (int i = 31; i >= 0; i--) {

            int bit = (num >> i) & 1;

            if (current.children[bit] == null) {

                current.children[bit] = new TrieNode();
            }

            current = current.children[bit];
        }
    }

    public int getMaxXor(int num) {

        TrieNode current = root;

        int answer = 0;

        for (int i = 31; i >= 0; i--) {

            int bit = (num >> i) & 1;

            int opposite = 1 - bit;
```

```
            if (current.children[opposite] != null) {
                answer |= (1 << i);
                current = current.children[opposite];
            } else {
                current = current.children[bit];
            }
        }
        return answer;
    }
}
```

---

47. Common Binary Trie Problems

Easy

- Store Binary Numbers

Medium

- Maximum XOR Pair
- Minimum XOR Pair

Hard

- LeetCode 1707
- Offline XOR Queries
- Network Routing

- IP Prefix Matching

---

48. Summary Comparison

Trie Type	Children	Best Use Case
Standard Trie	26	Lowercase words
HashMap Trie	Dynamic	Unicode / Dynamic alphabet
Compressed Trie	Variable	Memory optimization
Ternary Search Trie	3	Hybrid search
Binary Trie	2	XOR and bitwise problems

Interview Takeaways

- **Array Trie** is the default choice for lowercase English alphabets.
- **HashMap Trie** is ideal when the character set is dynamic or very large.
- **Compressed (Radix) Trie** reduces memory by merging single-child paths.
- **Binary Trie** is the go-to data structure for XOR-based interview problems.
- Recognize the phrase **"maximize XOR"** as a strong hint to use a Binary Trie.
